

KNUST · REGISTRY · INTERN PROJECT

Presenter's Handbook

Student Management System

Everything you need to present this project with confidence: timing plan, speaking guide, demo script, code explained in plain English, a question-and-answer bank, self-quiz and checklists.

SOURCE CODE

<https://github.com/acheampongmaamary/student-management-system>

Companion to: Student_Management_System_Presentation.pptx (16 slides, about 13 minutes)

Contents

1. Start here	3
2. Your 13-minute game plan	4
3. Slide-by-slide speaking guide	6
4. Live demo script and backup plan	7
5. The project at a glance	9
6. The code, explained	10
7. Data and API reference	13
8. Honest assessment: strengths and limits	15
9. Question and answer bank	17
10. Self-quiz	19
11. Troubleshooting cheat sheet	20
12. Presenting well: tips and checklists	21
13. Glossary and quick reference	22

GOOD TO KNOW

This handbook is a **companion** to the slide deck. The deck has a short speaker script in each slide's notes. This document goes deeper: it explains the project well enough that you can answer questions beyond the slides.

1. Start here

You are presenting a real, working full-stack web application. You do **not** need to be an expert to present it well. You need to understand the story, know the main ideas, and be honest about what is simple and what could be improved. This handbook helps with all three.

Your project in one sentence

TIP

"I built a web app where registry staff can add, view, edit and delete student records, with smart checks that stop mistakes before the data is saved to a cloud database."

The 3 things your audience must remember

- 1 **What it does:** full CRUD (Create, Read, Update, Delete) for student records, with no page reloads.
- 2 **How it is built:** plain HTML, CSS and JavaScript in the browser; a tiny Express server that delivers the files; and Supabase as the cloud database.
- 3 **What you learned:** file paths must match, asynchronous code needs care, and browser checks are not real security.

How to use this handbook

When	Read	Why
A few days before	Sections 5, 6, 7	Understand the project deeply
The day before	Sections 2, 3, 4, 9	Rehearse timing, the demo and likely questions
One hour before	Sections 11, 12	Run the checklists, fix problems early
If you get stuck	Sections 11, 13	Troubleshooting and glossary

2. Your 13-minute game plan

The slides are designed for **about 13 minutes**, which leaves a safety margin inside a 10 to 15 minute slot. Aim for the times below. They are targets, not strict limits. The times written at the top of each slide's speaker notes match this table.

#	Slide	Target	Key message to land
1	Title	0:20	Who you are and what you built
2	Roadmap	0:20	Tell them what to expect
3	Problem and goal	0:45	Why a registry needs this
4	Features (CRUD)	0:45	Four operations, no page reloads
5	Live demo	1:30	Show it working for real
6	Tech stack	0:50	Each tool has one clear job
7	Architecture	1:15	Server delivers; browser talks to the database
8	Project structure	0:30	Small, easy-to-navigate codebase
9	server.js	1:00	About 20 lines; static files plus one route
10	HTML and CSS	0:50	Semantic HTML, CSS variables, Grid
11	script.js and the API	1:15	GET, POST, PATCH, DELETE with fetch
12	Validation	0:50	Mistakes are caught before saving
13	How Edit works	0:50	One variable switches add and update
14	Lessons learned	1:00	Honest reflection, including security
15	Future improvements	0:45	A realistic roadmap
16	Thank you and Q&A	0:20	Repo link, how to run it, questions

Total: about 13 minutes 5 seconds, plus questions.

If you are running long: the cut list

- **Cut first:** slide 8 (project structure). Say one sentence and move on (saves 30 s).
- **Cut second:** slide 10 (HTML and CSS). Mention only semantic HTML and CSS variables (saves 30 s).
- **Shorten:** the demo. Show only add and age auto-fill (saves 45 s).
- **Never cut:** slide 7 (architecture), slide 14 (lessons learned) and the Q&A.

If you are running short

- Extend the live demo: try a duplicate ID, an invalid level, then edit and delete a record.
- Open **script.js** in your editor and read the submit handler aloud, step by step (Section 6).

TIP

Practise **out loud with a timer** at least twice. Most people speak faster when nervous, so if your practice run is 12 minutes you will probably land near 11 on the day.

3. Slide-by-slide speaking guide

Use this as a memory jogger. Do **not** read the slides aloud: the audience can read faster than you can talk. Say the ideas in your own words and use the slide as a visual aid.

Slide	Say	Do / point at
1 Title	Introduce yourself and the project in one sentence.	Pause. Smile. Mention the GitHub link is on screen.
2 Roadmap	Read the six items quickly. Say how long the talk is.	Do not explain each item yet.
3 Problem and goal	Paint a picture of a busy registry: many fields, many chances for typos and duplicates.	Point to the four number boxes: 4, 11, 13, 0.
4 Features	Walk across the four cards left to right: C, R, U, D. Mention the HTTP label on each.	Stress "no page reloads" in the green bar.
5 Live demo	Switch to the browser (localhost:3000). Follow the demo script in Section 4.	Return to the slides to continue.
6 Tech stack	Use the analogy on each card: skeleton, clothes, brain, engine, delivery person, filing cabinet.	Say you chose plain JavaScript on purpose, to learn the fundamentals.
7 Architecture	Trace arrow 1 (browser asks server for files) then arrow 2 (browser talks to Supabase).	Key line: "The data never passes through my server."
8 Structure	One sentence: server.js is the back end, everything in public/ is the front end.	Optional: skip if short on time.
9 server.js	Go through the four numbered steps on the right: import, static files, route, listen.	Mention the /public path lesson (it comes back on slide 14).
10 HTML and CSS	Left: smart input types and semantic tags. Right: colour variables and CSS Grid.	Point at the colour swatches.
11 script.js	Use the table as a cheat sheet. Then explain async/await using the code box.	Say: "await means wait for the answer without freezing the page."
12 Validation	Pick two or three rules, not all six. The level-by-programme rule is the most interesting.	Show the age snippet last.
13 Edit flow	Follow the five boxes. Then explain the editingStudentId variable at the bottom right.	Key line: "null means adding, a value means editing."
14 Lessons	Talk about lessons 1, 4 and 5 in detail; mention 2 and 3 briefly.	Be honest: it shows maturity.
15 Future	Choose one item from each column and say why it matters.	Mention the challenges in docs/INTERN_ONBOARDING_GUIDE.md.
16 Close	Thank the audience. Say how to run it in four commands. Invite questions.	Keep the slide up during Q&A.

4. Live demo script and backup plan

A live demo is the most memorable part of a talk, and the part most likely to go wrong. Prepare for both.

Before you start (do this 15 minutes earlier)

- ☐ Open a terminal in the project folder and run **npm start**. You should see the "up and running" banner.
- ☐ Open **http://localhost:3000** in your browser. The status line should say how many records are in the register.
- ☐ Check you have internet. The data lives in Supabase, so the table only loads online.
- ☐ Zoom the browser to 110 to 125 percent so the back row can read it (Ctrl and +).
- ☐ Close notifications, chat apps and extra browser tabs.
- ☐ Have the slide deck open on the demo slide (slide 5) so you can switch back easily.

The 90-second demo

- 1 **Show the page.** "This is the registry ledger. The table at the bottom loads from the cloud database."
- 2 **Type a name** in First name and Surname.
- 3 **Student ID:** type letters first, then digits. "Letters are removed automatically, and it stops at 8 digits."
- 4 **Date of birth:** pick a date. "Age fills in by itself." Try a future date to show the error message.
- 5 **Programme:** type "Bsc Computer" and pick it. Open Level: "500 and 600 are greyed out because this is a four-year degree." Then switch to **Doctor Of Medicine** and show 600 is available.
- 6 **Add to register.** Point to the new row and the status message.
- 7 **Edit** the row: change the phone number, click **Update student**.
- 8 **Delete** the demo row and confirm. "The row disappears without the page reloading."

WATCH OUT

Use **made-up demo data** (for example "Ama Serwaa", ID 20345678). Do not show other people's real details on a projector, and always delete your demo record afterwards.

If something goes wrong

Problem	What to do
Table says "Could not load student records"	Probably no internet or Supabase is paused. Stay calm, say so, and use the screenshots on slides 5 and 13.
Page looks unstyled (plain text)	The CSS did not load. Check the address is localhost:3000, restart npm start, and hard refresh (Ctrl + F5).
Browser can't connect	The server is not running or is on another port. Restart npm start and check the banner for the port.
Nothing happens when you click Add	A field is invalid. Look for the red underline or the browser tooltip and fix that field.

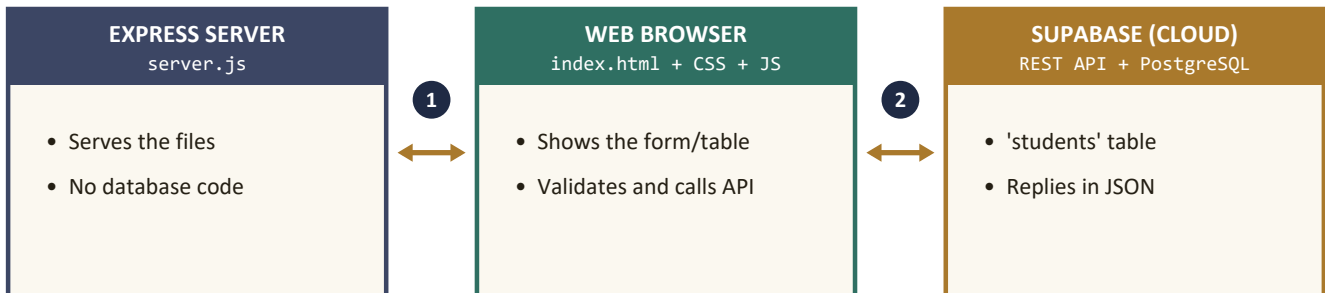
Problem	What to do
Demo completely fails	Say: "Let me show you from the slides instead," and continue. The audience cares more about your explanation than a perfect demo.

5. The project at a glance

What is it?

A **Student Management System** for a university registry. Staff use one web page to register students, see everyone in a table, change details, and remove records. Data is stored online in a Supabase (PostgreSQL) database.

How the three parts connect



- **Arrow 1:** you open localhost:3000. The browser asks Express for the page, the CSS and the JavaScript.
- **Arrow 2:** once loaded, **script.js** uses **fetch()** to read, save, update and delete students in Supabase.

Tech stack

Tool	Role (analogy)	Used for
HTML5	The skeleton	Page structure: form, inputs, table area
CSS3	The clothes	Registry look, CSS Grid layout, responsive rules
JavaScript	The brain	Clicks, validation, age maths, database calls
Node.js	The engine	Runs JavaScript on the computer
Express 5	The delivery person	Local web server for the files
Supabase	The filing cabinet	Cloud PostgreSQL database with REST API

File map

File	What it is
server.js	Starts the Express server on port 3000 (or the PORT variable)
index.html	The page: header, entry form, register table, script tags
public/css/style.css	All styling: variables, form, ledger table, mobile rules
public/js/script.js	All logic: validation, age, table rows, Supabase calls
package.json	Dependency list (Express) plus npm start and npm run dev
README.md and docs/	Written guides, including the intern onboarding guide

6. The code, explained

You do not need to memorise code. You need to **understand the idea** of each part and be able to say it in plain words. Each block below shows a short excerpt followed by a plain-English explanation.

A. server.js: delivering the files

server.js (simplified)

```
const express = require("express");
const path = require("path");
const app = express();
const PORT = process.env.PORT || 3000;

// Let the browser download /public/css/style.css etc.
app.use("/public", express.static(path.join(__dirname, "public")));

// Visiting the home page returns index.html
app.get("/", (req, res) => {
  res.sendFile(path.resolve(__dirname, "index.html"));
});

app.listen(PORT);
```

- **require** loads a library. **express()** creates the app that handles every visit.
- **express.static** is **middleware**: when the browser asks for a file inside **public**, Express finds it and sends it.
- **app.get("/")** is a **route**: it decides what to reply when someone visits the home page.
- **app.listen** opens the port. **process.env.PORT || 3000** means "use the PORT setting if present, else 3000".

B. script.js: talking to Supabase

script.js: create a student

```
const BASE_URL = `https://${PROJECT_REF}.supabase.co/rest/v1/students`;

const headers = {
  apikey: SUPABASE_ANON_KEY,
  Authorization: `Bearer ${SUPABASE_ANON_KEY}`,
  "Content-Type": "application/json",
  Prefer: "return=representation", // send the saved row back to me
};

async function insertStudentData(studentPayload) {
  const response = await fetch(BASE_URL, {
    method: "POST",
    headers,
    body: JSON.stringify(studentPayload),
  });
  if (!response.ok) throw new Error(await getErrorMessage(response));
  const data = await response.json();
  return data[0];
}
```

- **fetch** sends an HTTP request over the internet. **await** pauses **this function** until the answer arrives, but the page stays responsive.
- **JSON.stringify** turns a JavaScript object into text that can travel over the network.
- **response.ok** is false for errors (like 400 or 500). We then **throw** an error, which the caller catches and shows as a message.
- The other three functions work the same way: GET reads, PATCH updates (**?student_id=eq.<id>**), DELETE removes.

C. script.js: what happens when you click "Add to register"

- 1 **event.preventDefault()** stops the browser from reloading the page.
- 2 **Duplicate check:** is this ID already in the **registeredIds** list (unless we are editing this very student)?
- 3 **Level check:** is the chosen level higher than the programme allows? If so, set a custom error message.
- 4 **form.reportValidity()** asks the browser to check required fields and show tooltips. If invalid, stop here.
- 5 **Build the payload:** read each input, trim spaces, turn an empty middle name into **null**, turn Level into a number.
- 6 **Save:** if **editingStudentId** is not null, send PATCH (update). Otherwise send POST (create).
- 7 **Update the screen:** add or replace the table row, update the ID list, reset the form, show a status message.
- 8 **finally** re-enables the button whether the save worked or failed.

D. Age and level rules

script.js: age and maximum level

```
function calculateAge(dobValue) {
  const dob = new Date(dobValue);
  const today = new Date();
  let years = today.getFullYear() - dob.getFullYear();
  const monthDiff = today.getMonth() - dob.getMonth();
  // birthday not reached yet this year? subtract one
  if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < dob.getDate())) {
    years--;
  }
  return years >= 0 ? years : "";
}

function maxLevelFor(programmeValue) {
  const years = PROGRAMME_DURATIONS[normalizeProgramme(programmeValue)]
    ?? DEFAULT_PROGRAMME_YEARS; // unknown programme -> 4 years
  return years * 100;           // 4 years -> 400
}
```

E. Drawing the table safely

appendRowToTable(student) creates a **div** for the row, then a **span** for each value, and adds Edit and Delete buttons. It uses **span.innerText = value** rather than **innerHTML**. That is deliberate good practice: **innerText treats the value as plain text**, so a malicious name such as **<script>...** can never run as code (this protects against XSS attacks).

GOOD TO KNOW

Every row is saved in a JavaScript **Map** called **studentRows**, keyed by student ID. When you update or delete a student, the code finds the exact row instantly with **studentRows.get(id)** instead of searching the whole table.

7. Data and API reference

Form field to database column

Form field	Database column	Rule in the app
First name	<code>first_name</code>	Required
Middle name	<code>middle_name</code>	Optional; empty becomes null
Surname	<code>surname</code>	Required
Student ID	<code>student_id</code>	Exactly 8 digits; no duplicates
Gender	<code>gender</code>	Female / Male / Other
Date of birth	<code>date_of_birth</code>	Required; cannot be in the future
Age	<code>age</code>	Calculated automatically (read-only)
Email	<code>email</code>	Required; must look like an email
Phone number	<code>phone_number</code>	Digits only, up to 10
Programme	<code>course</code>	Required; suggestions from a list
Level	<code>level</code>	100 to 600, limited by programme

The four API calls

Action	Method and URL	Body
Read all	GET <code>/rest/v1/students?select=*&order=student_id.asc</code>	none
Create	POST <code>/rest/v1/students</code>	student JSON
Update	PATCH <code>/rest/v1/students?student_id=eq.20345678</code>	changed fields as JSON
Delete	DELETE <code>/rest/v1/students?student_id=eq.20345678</code>	none

In `?student_id=eq.20345678`, **eq** means "equals". It tells Supabase which student to change or remove.

Example of one student as JSON

example record (made-up data)

```
{
  "first_name": "Ama",
  "middle_name": "Nyarko",
  "surname": "Serwaa",
  "student_id": "20345678",
  "gender": "Female",
  "date_of_birth": "2003-05-14",
  "age": 23,
  "email": "ama@st.knust.edu.gh",
  "phone_number": "0244123456",
  "course": "Bsc Computer Science",
  "level": 200
}
```

HTTP status codes you may see in DevTools

Code	Meaning
200 OK	Request worked and returned data
201 Created	A new record was saved
204 No Content	Success with nothing to return
400 Bad Request	Data sent was invalid or missing
401 / 403	Not allowed: check the API key and Row Level Security
404 Not Found	The URL or file does not exist
500	Something broke on the server side

8. Honest assessment: strengths and limits

Good presenters know where their project is strong **and** where it is still simple. Saying so confidently makes you more credible, not less.

Rules the app enforces

Rule	How it works
Student ID	Non-digits are stripped as you type; maximum 8 characters; pattern check on submit.
Duplicate ID	Compared with the list of IDs loaded on the page (and edit mode allows keeping your own ID).
Phone	Digits only, maximum 10 characters.
Date of birth	A future date sets a custom error; Age is calculated from the date.
Level	Programme sets a maximum (4 years: 400, Architecture 5: 500, Pharm D and MB ChB 6: 600). Options above it are disabled.

Strengths

- + Small, readable codebase with no heavy framework: easy to learn from.
- + Clear separation: server delivers files, browser handles logic, Supabase stores data.
- + Good user experience: instant feedback, automatic age, programme-aware levels, no page reloads.
- + Safe table rendering with **innerText**, and consistent error handling with try, catch and finally.

Limitations (say these before someone asks)

- **All checks run in the browser.** Someone could bypass them. The database should also enforce rules (unique student ID, valid values).
- **The duplicate check only knows IDs loaded on the page.** Two people adding the same ID at the same moment could both succeed unless the database has a unique constraint.
- **No login.** Anyone who can reach the page can change records. Production use needs staff authentication and Row Level Security.
- **Age is stored.** A stored age becomes outdated as time passes; it is more accurate to calculate it from the date of birth when displaying.
- **The API key and project settings are written in script.js.** The anon key is designed to be public, but settings are better placed in configuration.
- **No automated tests** yet. Everything was tested by hand.
- **Everything loads at once.** With thousands of students the table needs pagination and search.
- The Supabase JavaScript library is included in **index.html** but the app calls the REST API directly with **fetch**, so it is currently not needed.

TIP

A good answer pattern: "Right now it does X. For production I would add Y because Z." It shows you understand the real-world gap.

9. Question and answer bank

These are the questions you are most likely to be asked, with short answers you can adapt. If you do not know an answer, see the tip at the end of this section.

Why didn't you use React or another framework?

I wanted to learn the fundamentals first: HTML structure, CSS layout, and JavaScript events and fetch. Frameworks build on those, so understanding them makes frameworks easier later.

What is Supabase and why did you choose it?

Supabase is a hosted PostgreSQL database that automatically provides a REST API. It meant I could store real data in the cloud without writing and hosting my own database server.

Why does your Express server not talk to the database?

Supabase already offers an API the browser can call directly, so the server only needs to deliver the files. That kept the server to around 20 lines. In a bigger system I might route requests through my own server for extra control.

Is it safe to have the API key in the JavaScript file?

The Supabase **anon** key is designed to be public, but it is only safe if Row Level Security policies are turned on so that users can only do what they are allowed to. The secret **service_role** key must never be in front-end code.

What is REST? What do GET, POST, PATCH and DELETE mean?

REST is a common style for web APIs that uses standard HTTP methods. GET reads data, POST creates it, PATCH changes part of a record, DELETE removes it. They map directly onto Read, Create, Update and Delete.

What does `async/await` do?

Network requests take time. **await** pauses that function until the answer arrives, but the browser keeps responding to the user. It lets asynchronous code read like normal top-to-bottom code.

How do you stop duplicate student IDs?

The page keeps a list of IDs it has loaded and checks new IDs against it. The honest limit is that this only covers the loaded data; the safest fix is also a unique constraint on `student_id` in the database.

How does the level limit work?

Each programme has a duration in years. Maximum level is years times 100. When you choose a programme, options above the maximum are disabled, and the same check runs again on submit. Unknown programmes default to four years.

How is age calculated?

Current year minus birth year, then subtract one if the birthday has not yet happened this year.

Why not store only the date of birth and calculate age when needed?

That would be better, because a stored age goes out of date. I stored it to display it simply in the table; it is on my improvement list.

What happens if the internet is down?

The page still opens because the files come from the local server, but loading and saving fail and a red error message appears in the status line. There is no offline mode.

How did you test it?

Manually: adding, editing and deleting records, and trying invalid inputs. I have not written automated tests yet; the age and level functions would be the first to test.

Why use div elements for the table instead of a real table?

I used CSS Grid to get a flexible ledger layout. A semantic HTML table would be better for accessibility and screen readers, so that is a good improvement.

How would you add a search feature?

Add a search input and, on each keystroke, hide table rows whose text does not include the search words. The **studentRows** map already gives me access to every row.

How would you deploy it?

Host the Express app on a service such as Render, Railway or Azure App Service, or serve the static files from Netlify or Vercel. Supabase is already hosted online.

What was the hardest part?

Choose your own true answer. Good candidates: understanding async code and error handling, keeping the table and database in sync without reloading, or debugging the file path (the CSS returning a 404).

What would you do differently?

Add authentication and database rules earlier, write tests for the validation functions, and keep configuration out of the source code.

TIP

If you don't know: "That's a good question. I haven't explored that yet, but my approach would be to ... and I would check the documentation." Never guess; a calm, honest answer is respected.

10. Self-quiz

Cover the answers and try to respond out loud. If you can answer most of these, you know the project well enough to present it.

#	Question	Answer
1	What does CRUD stand for?	Create, Read, Update, Delete.
2	Which HTTP method creates a record? Which one updates part of a record?	POST creates; PATCH updates part of a record.
3	What is the job of server.js?	To start an Express server that delivers index.html, the CSS and the JavaScript to the browser.
4	Why must the server expose the folder under /public?	Because index.html asks for /public/css/style.css and /public/js/script.js. If the paths don't match, the browser gets a 404.
5	What does await do?	Pauses the function until the promise finishes, without freezing the page.
6	What does editingStudentId = null mean?	The form is in add mode. A value means the form is editing that student.
7	Why is innerText safer than innerHTML here?	It treats values as plain text so injected HTML or scripts cannot run (prevents XSS).
8	What is the maximum level for Architecture? For Medicine?	500 for Architecture (5 years); 600 for Doctor of Medicine (6 years).
9	What does ?student_id=eq.123 mean?	A Supabase filter: only the row where student_id equals 123.
10	Name two limitations of the project.	For example: checks only run in the browser, no login, no automated tests, duplicate check only knows loaded IDs.

11. Troubleshooting cheat sheet

Symptom	Likely cause	Fix
npm or node not recognised	Node.js is not installed or the terminal needs a restart	Install Node.js from nodejs.org , reopen the terminal, check with <code>node -v</code>
EADDRINUSE address already in use	Another program is using the port	Stop it, or run on another port: <code>\$env:PORT=4000; npm start</code> (PowerShell)
Page has no styling	CSS file returned 404	Open DevTools (F12) and Network tab; check the URL is under <code>/public</code> ; restart the server
Changes to files do not show	Browser cache	Hard refresh: <code>Ctrl + F5</code> , or run <code>npm run dev</code> for auto-restart
"Could not load student records"	No internet, Supabase paused, wrong key or blocked by policy	Check internet; open the Supabase dashboard and resume the project if paused; check the key and Row Level Security
Add button does nothing	A field fails validation	Look for red underline or tooltip; check ID has exactly 8 digits and level is allowed
"This student ID is already registered"	The ID exists in the table	Use a different ID or edit the existing record
Error 401 or 403 from Supabase	Key invalid or policies block the action	Re-check anon key; review Row Level Security policies

Using the browser's DevTools (press F12)

- **Console tab:** red text shows JavaScript errors. The app logs failures such as "Failed to save student".
- **Network tab:** choose Fetch/XHR and reload. Click a request to see the URL, status code, the JSON you sent and what came back.
- **Sources tab:** open `script.js`, click a line number to set a **breakpoint**, then trigger the action. Execution pauses so you can inspect variables.
- **Elements tab:** see the live HTML, including table rows created by JavaScript.

12. Presenting well: tips and checklists

Speaking tips

- **Slow down.** Nervous speakers rush. Breathe between slides and pause after key sentences.
- **Don't read the slides.** Look at the audience and use the slide as a prompt.
- **Use stories and analogies.** The deck's analogies (skeleton, brain, delivery person) are easy to remember.
- **Point at what you are talking about** (the arrows on slide 7, the buttons in the demo).
- **Explain jargon once.** Terms like fetch or middleware deserve one plain-English sentence.
- **Mistakes are fine.** If you lose your place, glance at the slide, take a breath and say "Let me come back to that."
- **Repeat questions** before answering so everybody hears them and you gain thinking time.

The day before

- ☐ Run through the whole presentation out loud, with a timer, at least twice.
- ☐ Clone the repository fresh and check it runs: **git clone, npm install, npm start.**
- ☐ Confirm the Supabase project is active (free projects can pause after a period of inactivity).
- ☐ Copy the slide deck (.pptx) and this PDF to a USB drive or cloud folder as backup.
- ☐ Fill in your name on slide 1 and in the speaker notes.
- ☐ Prepare your own answer for "What was the hardest part?".

One hour before

- ☐ Charge your laptop and bring the charger and any display adapter (HDMI or USB-C).
- ☐ Start the server and open localhost:3000. Confirm the table loads.
- ☐ Check the projector or screen works with your laptop. Increase browser zoom.
- ☐ Turn off notifications and close unnecessary apps.
- ☐ Open the deck in Presenter View so you can see your speaker notes.
- ☐ Water nearby. Take three slow breaths. You know this project.

Right after

- ☐ Delete any demo records you added so the database stays clean.
- ☐ Write down questions you could not answer; they make great next learning tasks.
- ☐ Share the GitHub link with anyone who asked for it.

13. Glossary and quick reference

Term	Plain-English meaning
API	A set of rules that lets two programs talk to each other.
Async / await	Do slow work (like a network request) in the background and continue when it finishes, without freezing the page.
BaaS	Backend-as-a-Service: a cloud platform (like Supabase) that gives you a database and API without building a server.
Browser DevTools	Built-in tools (F12) for inspecting HTML, errors and network requests.
CDN	A network of servers that delivers common files (like a JavaScript library) quickly.
CRUD	Create, Read, Update, Delete: the four basic data operations.
CSS Grid	A CSS layout system that arranges items into rows and columns.
CSS variable	A named value (like <code>--teal</code>) defined once and reused everywhere.
DOM	The browser's live, tree-shaped model of the page that JavaScript can change.
Endpoint	A specific URL of an API that does one job.
Event listener	Code that waits for something to happen (click, typing) and then runs.
Express	A small Node.js library for building web servers.
fetch()	The JavaScript function that sends an HTTP request.
HTTP method	The verb of a request: GET, POST, PATCH, DELETE.
JSON	A lightweight text format for data, like <code>{"name": "Ama"}</code> .
Middleware	Code that runs between a request arriving and the final reply.
Node.js	A program that runs JavaScript outside the browser.
npm	Node's package manager: installs libraries listed in <code>package.json</code> .
Payload	The data you send in the body of a request.
PostgreSQL	A popular, powerful open-source database.
REST	A style of web API based on URLs and standard HTTP methods.
Route	A rule that says what to reply for a given URL.
Row Level Security	Database rules that control which rows each user may read or change.
SPA	Single-page application: updates the page in place instead of reloading.
XSS	An attack that sneaks malicious script into a page; avoided by treating input as plain text.

Quick reference

commands

```
# Run the app
npm install          # first time only
npm start            # starts http://localhost:3000
npm run dev          # restarts automatically when files change

# Another port (PowerShell)
$env:PORT=4000; npm start

# Get the code
git clone https://github.com/acheampongmaamary/student-management-system.git
```

Repository: <https://github.com/acheampongmaamary/student-management-system>

Also in the project: **README.md** (overview and setup) and **docs/INTERN_ONBOARDING_GUIDE.md** (first-week roadmap and practice challenges).

You built this. You understand it. Go and show it with pride.